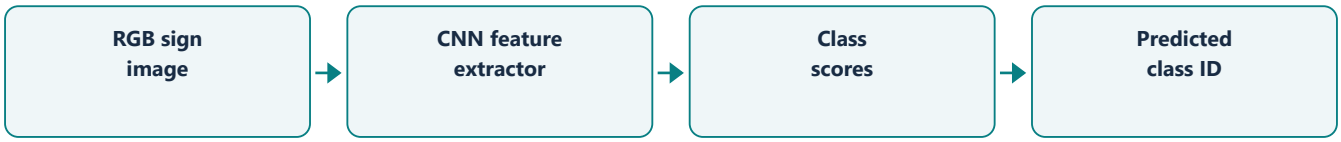
01 / Project at a glance

Traffic-sign classification and corruption robustness
A practical guide to the current CNNs, the concepts behind their layers, and the experiment pipeline. Built from the local Python implementation, not from a generic CNN template.

Source snapshot: 01 November 2026. GTSRB is the default; BTSD and CTSD are separate supported datasets. All three models currently use ReLU.



The network classifies an image; it does not locate signs in a road scene.

Model	What changes	What it tests
Baseline	Three convolution blocks and a dense classifier; no BatchNorm.	A standard reference CNN.
AdaBN	Adds four BatchNorm layers and target-statistic adaptation.	Whether target-domain BN statistics help under corruption.
ACO	Adds 128 channel weights selected by an ant-colony search.	Experimental feature reweighting alongside Adam training.

One experiment, not 70 training runs

Choose one model and one dataset. Train on clean training images, or load a checkpoint. Evaluate the clean test split, then evaluate 14 corruption types at five severities: **70 corrupted conditions plus one clean condition**. AdaBN adapts separately for each corrupted condition.

Reading map

Pages	Topics
2–5	Datasets, preprocessing, convolution, layer shapes, ReLU, pooling and dropout.
6–9	Batch Normalization, AdaBN and the exact ACO implementation.
10–13	Loss, gradients, Adam, corruption protocol and evaluation metrics.
14–17	Code map, commands, experiment limits, glossary and references.

Scope
This is image classification, not object detection, image restoration, or a verified autonomous-driving system. The networks are custom CNNs trained from scratch, not pretrained ResNet or YOLO models.

02 / Datasets and image preparation

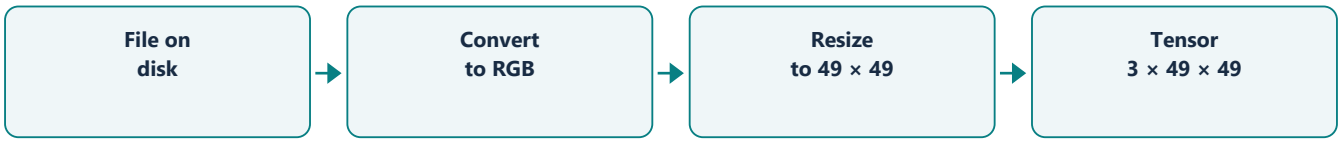
Dataset	Classes	Train	Test	Test classes present
GTSRB	43	39,209	12,630	43
BTSD	62	4,591	2,534	53
CTSD	58	4,931	1,233	57

Counts above were read from the local dataset loaders while generating this PDF. They describe this checkout, not every possible release of these datasets.

Where each split comes from

GTSRB, Germany: reads Train.csv and Test.csv, using native IDs 0–42.
BTSD, Belgium: reads training/testing class folders and their semicolon-separated CSV annotations, using IDs 0–61.
CTSD, China: reads annotations.csv and creates a stratified 80/20 split with random_state=42, using IDs 0–57. Stratification tries to preserve class proportions.

Separate label spaces
The datasets are not merged. A class ID is meaningful only inside its own dataset. The final layer has K=43, 62 or 58 outputs; a missing test class does not shrink that layer.



A batch of B images has shape B × 3 × 49 × 49.

What the numbers mean

RGB has three colour channels. ToTensor converts the usual 8-bit RGB pixels into floating-point values in [0, 1]; for example, 255 becomes 1 and 128 becomes about 0.502. This input scaling is different from Batch Normalization inside the network.

What preprocessing does not do

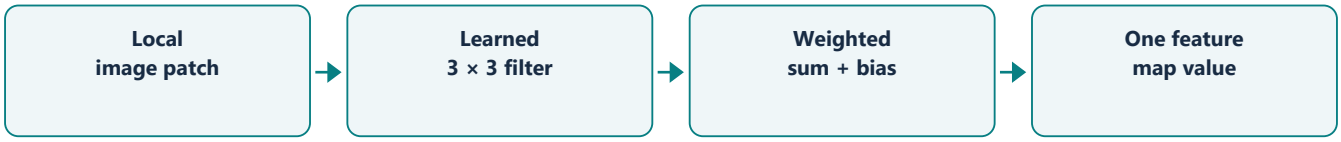
There is no additional channel mean/std normalization, bounding-box crop, aspect-ratio-preserving padding, or random training augmentation in the default path. Resizing a non-square source image to a square can distort its proportions.

Caching and split caution

The loader caches transformed tensors and original PIL images in memory to avoid repeated file reads. PIL is an image representation, not a neural-network layer. CTSD uses a row-level split: related filename variants may cross splits. Their provenance should be checked before claiming independent generalization.

03 / How a convolution learns features

A **convolutional neural network** uses small learned filters to inspect local image regions. Each filter produces a **feature map**: a spatial grid of responses. The same filter weights are reused across the image, rather than learning a separate detector for every pixel location.



The filter slides across spatial positions. Its weights are learned through backpropagation.

A filter sees every input channel

In the first layer, one output filter has shape 3 × 3 × 3: three input colour channels and a 3 × 3 spatial window. It combines all 27 values and adds one bias. With 32 output filters, the layer creates 32 feature maps, each initially 49 × 49.

```
One output value = sum(filter weights × input patch) + bias

Conv parameters = C_out × (C_in × kernel_height × kernel_width + 1)

First layer = 32 × (3 × 3 × 3 + 1) = 896 parameters
```

Kernel, stride and padding

Term	Meaning here
Kernel = 3 × 3	The spatial window inspected by each convolution.
Stride = 1	Move the window one spatial position at a time.
Padding = 1	Add a one-pixel zero border so these convolutions preserve height and width.
Channels 32 → 64 → 128	The number of learned feature maps increases as spatial size decreases.

```
Output size = floor((input + 2 × padding - kernel) / stride) + 1

Convolution: (49 + 2 - 3) / 1 + 1 = 49

Pooling: floor((49 - 2) / 2) + 1 = 24
```

What might the filters represent?

Early layers can learn edges and colour transitions; later layers can combine responses into shapes and sign-related patterns. That is an intuition, not a verified interpretation of this project's individual filters. A channel is not guaranteed to represent one human-readable concept.

Feature extraction is learned
No hand-coded triangle detector or digit recognizer is supplied. Labelled examples and the loss function guide the filters toward useful classification features.

04 / The baseline CNN, layer by layer

Source: models/baseline/model.py:18. B is batch size; K is dataset class count. Shapes below omit B. All convolutions use kernel 3, stride 1, padding 1. All pooling uses kernel 2, stride 2.

Stage	Operations in actual order	Output shape
Input	RGB tensor	3 × 49 × 49
Block 1	Conv 3→32; ReLU; MaxPool	32 × 24 × 24
Block 2	Conv 32→64; ReLU; Dropout2d(0.2); MaxPool	64 × 12 × 12
Block 3	Conv 64→128; ReLU; MaxPool	128 × 6 × 6
Flatten	Rearrange all feature values into a vector	4,608
Hidden head	Linear 4,608→512; ReLU; Dropout(0.5)	512
Output head	Linear 512→K; no output activation	K logits

Why 4,608 inputs to the classifier?

Pooling rounds down: 49 → 24 → 12 → 6. The last block has 128 maps of size 6 × 6, so flattening gives 128 × 6 × 6 = **4,608 values per image**. Flatten changes the arrangement, not the values or the number of trainable parameters.

Trainable component	Parameters for GTSRB
Convolution 1 / 2 / 3	896 / 18,496 / 73,856
Linear 4,608 → 512	2,359,808
Linear 512 → 43	22,059
Total	2,475,115

Where the capacity is

The first dense layer contains about 95% of the baseline's trainable parameters. Most parameter storage is therefore in the classifier, not the convolutional filters. For any K, the baseline/ACO total is 2,453,056 + 513K.

Keep the intended image size
The classifier expects exactly 4,608 features. Changing --target-size to an arbitrary size can cause a matrix-shape error. The CLI option alone does not make the architecture resolution-independent.

05 / ReLU, pooling, dropout and logits

ReLU: allow nonlinear decisions

ReLU(x) = max(0, x). For inputs [-2, 0, 3], the outputs are [0, 0, 3]. Without nonlinear activations between linear operations, the network cannot express the same rich decision boundaries. All three current models use ReLU; the baseline and ACO previously used GELU.

Max pooling: retain a strong local response

Each 2 × 2 window is reduced to its maximum. For [[1, 5], [3, 2]], the result is 5. This reduces spatial computation and gives limited tolerance to small changes in position. It also discards spatial detail; pooling is not guaranteed robustness to large shifts.

Dropout: avoid relying on one response

During training, Dropout2d(p=0.2) in block 2 randomly zeroes whole feature channels for each sample. Dropout(p=0.5) after the hidden dense layer randomly zeroes individual activations. Surviving activations are scaled by 1/(1−p). This encourages the network not to depend on a single feature.

Operation	Training	Normal evaluation
ReLU / MaxPool / Linear	Same forward operation	Same forward operation
Dropout / Dropout2d	Randomly suppress responses	Disabled; activations pass through
BatchNorm, AdaBN only	Uses current batch; updates running statistics	Uses stored running statistics

Dense layers and class scores

The first Linear layer mixes the 4,608 extracted features into 512 hidden values. The last Linear layer produces one **logit**, or raw score, for each class. These scores can be negative and do not sum to one.

```
Illustrative logits = [0.2, 2.1, -0.4]
argmax(logits) = 1 → predict class ID 1
softmax(logits) ≈ [0.121, 0.812, 0.067]
```

The three-class example illustrates scoring only; the real heads have 43, 62 or 58 outputs. Prediction uses argmax directly. Softmax is useful for explaining probabilities, but the implementation passes raw logits to CrossEntropyLoss.

Mode and gradients are different controls
model.eval() changes dropout/BN behaviour; it does not itself disable gradients. The test methods also use torch.inference_mode(). AdaBN uses no_grad() while deliberately enabling only BN statistic updates.

06 / Batch Normalization, properly explained

Batch Normalization (BN) normalizes intermediate features, then applies a learned scale and shift. It often makes optimization better behaved, but it is not a promise of better accuracy and it does not normalize the original image pixels.

```
x_normalized = (x - mean) / sqrt(variance + epsilon)

BN(x) = gamma × x_normalized + beta

Here: epsilon = 0.00001; gamma and beta are learned per channel
```

A small numerical example

For one feature with batch values [2, 4, 6, 8], the mean is 5 and the biased variance is 5. Ignoring epsilon for readability, normalized values are about [-1.342, -0.447, 0.447, 1.342]. If gamma=2 and beta=1, the outputs become about [-1.683, 0.106, 1.894, 3.683]. Gamma and beta let the model keep the scale/offset useful for its task.

Layer in this project	Where statistics are computed	Preserved shape
BatchNorm2d(32/64/128)	For each channel, across images and spatial positions: B × H × W values.	B × C × H × W
BatchNorm1d(512)	For each hidden feature, across the B images in the dense-layer batch.	B × 512

Training versus evaluation

Training uses current-batch statistics and updates running mean/variance buffers. Normal evaluation uses those stored buffers instead. The default update is running_new = 0.9 × running_old + 0.1 × batch_statistic. This BN momentum is not Adam's momentum setting.

PyTorch normalizes a training batch using its biased variance estimate, but updates running variance with an unbiased estimate. The numerical example above describes the normalization step; it is not the exact running-variance update.

Parameters are not the same as buffers

Parameters: gamma and beta are learned by Adam during supervised training.
Buffers: running mean, running variance and batch counters are saved in state_dict but are not trained through gradients. AdaBN changes these statistics, not gamma/beta.

Batch size matters
The classifier's BatchNorm1d needs more than one example when using batch statistics. A one-image final training or adaptation batch can crash. Small or unrepresentative batches can also make the estimated statistics unreliable.

07 / AdaBN architecture and adaptation

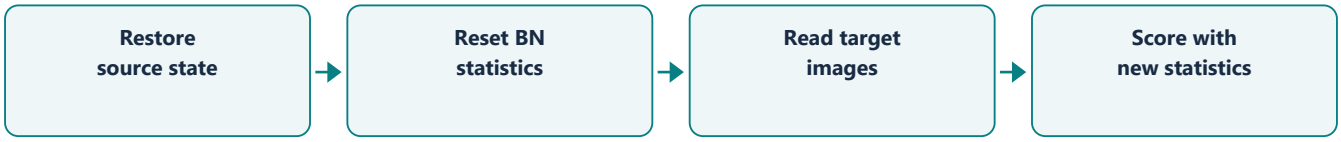
Source: models/adabn/model.py:23 and adapt_bn():181. This is a separate model class, not a flag that inserts BN into a trained baseline. Its channel counts and spatial sizes match the baseline.

```
Conv 3→32 → BN2d(32) → ReLU → MaxPool
Conv 32→64 → BN2d(64) → ReLU → Dropout2d(0.2) → MaxPool
Conv 64→128 → BN2d(128) → ReLU → MaxPool
Flatten → Linear 4608→512 → BN1d(512) → ReLU
Dropout(0.5) → Linear 512→K → logits
```

For GTSRB this has **2,476,587 trainable parameters**: 1,472 more than the baseline. BN adds two learned values per channel/feature: 2 × (32+64+128+512). Running statistics are additional buffers, not counted here.

Why adapt?

A **domain shift** is a change in the data distribution. Fog can alter brightness and contrast, so the internal feature statistics differ from those seen during clean training. AdaBN re-estimates BN statistics from the target images to account for that shift. It does not remove fog from the input.



Repeated independently for every corruption/severity pair; no target labels are used in adaptation.

Exact procedure in this implementation

1. Reset the BN running statistics and set momentum=None: cumulative averaging over batches.
2. Put only BN layers into training behaviour. Keep dropout and other layers in evaluation mode.
3. Run the corrupted target loader once with no_grad(); ignore labels.
4. Return to evaluation mode, restore BN momentum to 0.1, and score the condition. No optimizer step occurs.

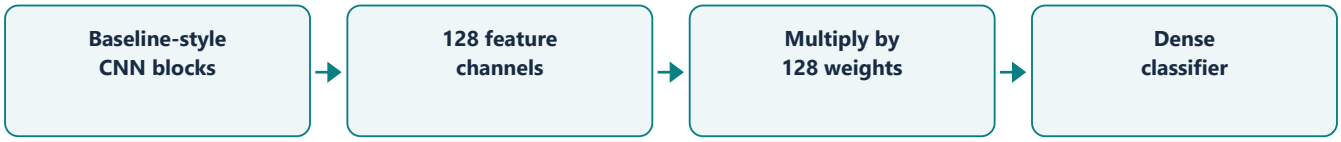
The evaluator restores the original trained state before each new condition, preventing one corruption's statistics from carrying into another. Clean evaluation uses source statistics without this adaptation pass. The source state is also restored after the benchmark.

> **Fair comparison**
> Baseline versus AdaBN changes both architecture and adaptation. To isolate adaptation, compare the same trained BN checkpoint with adaptation disabled versus enabled. This project's default run does not automatically produce that controlled ablation.

Deployment limit: this is batch/transductive test-time adaptation: target images are seen before scoring. It is not a verified one-image-at-a-time online method. See the AdaBN paper in the references for the underlying idea.

08 / ACO: a channel-weighted CNN

Source: models/aco/model.py:131. ACO means **Ant Colony Optimization**: a search method inspired by the reinforcement of useful trails. Here, artificial ants propose feature-channel masks. There are no literal ants and no additional image dataset involved.



The gate sits after the third pooling layer and before flattening.

```
features: B × 128 × 6 × 6

pheromone_gate: 1 × 128 × 1 × 1

gated_features = features × pheromone_gate

logits = classifier(flatten(gated_features))
```

What does one gate value do?

Each gate value is shared across every pixel in one channel and across all images. If a feature response is 4, a weight of 0.5 makes it 2; 1 leaves it 4; 1.5 makes it 6. This is global channel weighting, not an image-specific spatial attention map.

What is learned by which optimizer?

Quantity	How it changes	Saved in checkpoint?
Convolution and dense weights	Adam uses labelled training batches and gradients.	Yes, as parameters.
128 channel gate values	ACO search computes them from pheromones; Adam does not update them.	Yes, as a registered buffer.
Ant masks / pheromone search history	Temporary search state inside the optimizer object.	Not as a resumable search history.

Training schedule

Epoch 1: ordinary Adam training with the neutral all-ones gate.
Before epoch 2: run the ACO search once and copy the resulting gate into the model.
Epoch 2 onward: continue Adam training with that fixed gate.

During search, the model enters evaluation mode so dropout does not add random noise to candidate fitness. Training mode is restored afterward, including if search raises an exception. At least two epochs are needed; --epochs 1 never reaches ACO.

Reweighting is not pruning
The final weights are approximately 0.5–1.5, so channels remain present. Parameter count and layer shapes do not shrink. ACO supplements Adam; it does not replace gradient-based CNN training.

09 / The exact ant-colony search

Setting	Value	Meaning
Channels / ants / iterations	128 / 15 / 5	75 candidate masks are evaluated by default.
rho / alpha / beta / Q	0.15 / 1 / 1 / 1	Evaporation, pheromone importance, heuristic importance, deposit scale.
Initial pheromones tau	All ones	No channel initially preferred by pheromone.

1. Extract features and estimate desirability

Take one batch from the shuffled clean training loader. Extract B × 128 × 6 × 6 features. For each channel, calculate variance over batch and spatial dimensions, add 1e-6, and normalize by the largest variance plus 1e-6. This gives heuristic desirability eta. High variance is a heuristic, not proof of useful or noise-invariant features.

2. Sample candidate masks

```
score_i = tau_i^alpha × eta_i^beta

p_i = clip(score_i / (1 + score_i), 0.1, 0.95)

mask_i = 1 if a random uniform draw is below p_i; otherwise 0
```

Each ant makes one binary inclusion decision per channel. If fewer than 32 of the 128 channels are active, the code activates the highest-probability 64 channels as a fallback; any other already-selected channels remain active.

3. Score masks and update pheromones

```
fitness = correct predictions / images in that same batch

tau = (1 - rho) × tau

For each ant: tau += Q × fitness^2 × mask
```

The classifier scores masked features without gradient updates. Better masks deposit more pheromone on their selected channels. With fitness 0.8 and Q=1, each selected channel receives 0.64 from that ant. Evaporation reduces old pheromones before new deposits.

4. Convert pheromones into a continuous gate

```
gate_i = 0.5 + (tau_i - min(tau)) / (max(tau) - min(tau) + 1e-6)
```

The search tracks a best binary mask, but it returns the continuous gate derived from accumulated pheromones, not that best mask. If all pheromones are equal, the formula returns 0.5 for every channel.

What the objective actually proves
Fitness is clean accuracy on one labelled training batch. It is not held-out accuracy or corrupted-image robustness. A successful search or a completed training run does not establish a robustness improvement.

10 / Loss, predictions and backpropagation

This is **supervised multiclass learning**: each training image has one correct class ID. The model outputs K logits. Cross-entropy encourages the correct class to score higher than alternatives.

Softmax explains the scores

```
p(class i) = exp(logit_i) / sum_j exp(logit_j)

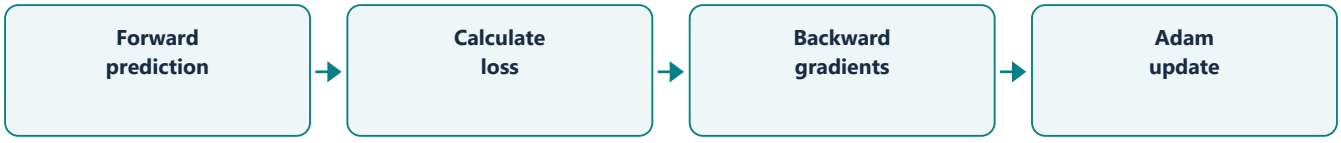
Loss for one image = -log(p(correct class))

Batch loss = mean of the per-image losses
```

If the model assigns the true class probability 0.8, the loss is about 0.223. If it assigns only 0.1, the loss is about 2.303. Thus a confident wrong prediction is penalized more strongly. Probabilities here explain the loss; the code uses a numerically stable CrossEntropyLoss directly on logits.

> **Do not add softmax before this loss**
> CrossEntropyLoss already includes the equivalent log-softmax calculation. Passing probabilities instead of logits changes the intended objective. Argmax of logits and argmax of their softmax give the same predicted class.

The learning cycle



Repeat over shuffled training batches for each epoch.

```
optimizer.zero_grad()

logits = model(images)

loss = criterion(logits, labels)

loss.backward()

optimizer.step()
```

What each step means

zero_grad: clear previous gradients because PyTorch otherwise accumulates them.
forward: produce class scores using the current weights.
loss: compare scores to the ground-truth IDs.
backward: use the chain rule to calculate how each trainable parameter affects the loss.
step: let Adam update those parameters.

Labels, gradients and adaptation

Labels guide ordinary training and ACO mask fitness. They are used for scoring test predictions, but AdaBN's adaptation pass ignores them. AdaBN updates running statistics through forward passes, not through backward() or optimizer.step().

11 / Adam and the training settings

Training setting	Current default
Optimizer / learning rate	Adam / 0.001
Adam beta1 / beta2 / epsilon	0.9 / 0.999 / 1e-8
Weight decay / loss	0 / CrossEntropyLoss
Epochs / training batch size	5 / 64
Training order / device	Shuffled / CUDA if available, otherwise CPU
Clean / corruption batch size	64 / 128 at the default CLI batch size

What Adam does

Adam keeps moving averages of gradients and squared gradients for each parameter. The first tracks direction; the second tracks recent gradient magnitude. Bias correction compensates for averages starting at zero. The update uses these estimates to scale the step. Adaptive step sizes do not eliminate the need to choose a learning rate.

```
m_t = beta1 × m_(t-1) + (1-beta1) × gradient
v_t = beta2 × v_(t-1) + (1-beta2) × gradient^2
m_hat = m_t / (1-beta1^t); v_hat = v_t / (1-beta2^t)
weight -= learning_rate × m_hat / (sqrt(v_hat) + epsilon)
```

Batch, iteration and epoch

A **batch** is one group of images. A normal training **iteration** performs one forward/backward/update cycle on that group. An **epoch** visits the training split once. With 39,209 GTSRB images and batch size 64, there are 613 training batches per epoch, including a smaller final batch.

Overfitting and model selection

Overfitting means improving performance on training examples without matching improvement on independent data. Dropout can reduce reliance on individual responses, but does not prevent overfitting by itself. This code saves the final epoch's weights: it has no validation split/loop or best-validation checkpoint selection.

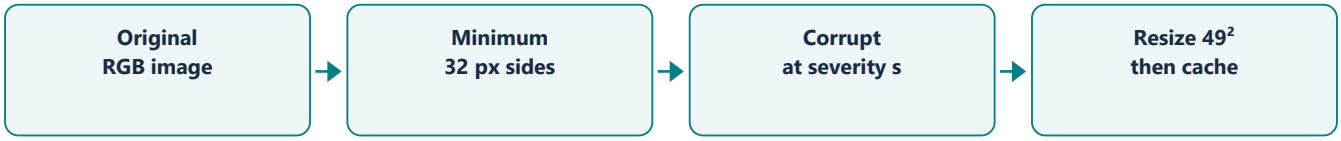
There is no learning-rate scheduler, early stopping, explicit class balancing, corruption augmentation, mixed-precision training, or centrally controlled random seed in the normal path. CTSD's split seed does not seed model initialization, shuffle, ACO or corruptions.

Five epochs are a default, not a proven optimum
Choose training duration using a validation protocol, not by repeatedly tuning against the final test set. The CLI exposes epochs and batch size, but not learning rate or ACO search settings.

12 / The corruption benchmark

After clean evaluation, the same trained model is tested under synthetic distribution shifts generated with imagecorruptions. Severity runs from 1 to 5. These are five settings per corruption, not five training epochs or an equal physical strength across all corruption types.

Group	Corruptions used	Intuition
Noise-like	gaussian_noise, impulse_noise, shot_noise	Random intensity changes or sensor-like noise.
Weather-like	snow, fog, frost	Synthetic weather/visibility effects.
Blur	motion_blur, zoom_blur, defocus_blur	Loss or smearing of spatial detail.
Appearance	brightness, contrast	Intensity and contrast shifts.
Digital / distortion	pixelate, elastic_transform, jpeg_compression	Coarse sampling, warping and compression artifacts.



Each condition is generated once and saved as a memory-mapped uint8 RGB array.

Why order matters

Blurring an original larger image and then reducing it is not identical to blurring an already tiny 49 × 49 image. The wrapper starts with the original PIL image. Any dimension below 32 is enlarged to meet the corruption library's requirement; after corruption the result is resized bilinearly to 49 × 49.

Baseline/ACO versus AdaBN

Baseline and ACO: predict with the trained source state and no target adaptation.
AdaBN: restore source state, adapt BN statistics on the target condition, then score it. This adds one forward pass over the target data per condition. Source state restoration keeps the 70 adapted conditions independent.

The evaluator caches resized corruptions under output/.corruption_cache/ and reuses them across models and AdaBN adaptation/scoring. Completed conditions survive interruption. The first run still generates images; later runs read them from disk. All 70 conditions at 49 × 49 require about 6.37 GB for GTSRB, 1.28 GB for BTSD and 0.62 GB for CTSD. Source metadata, resolution, code and library versions identify the cache.

Do not overstate the protocol
This is a selected 14-corruption benchmark. It is not the complete ImageNet-C protocol, does not train on ImageNet, and does not compute ImageNet-C reference-normalized error. Synthetic robustness is not evidence of road safety.

13 / How performance is measured

Accuracy: how many predictions are correct?

```
Accuracy (%) = 100 × correct predictions / total images
```

If 80 of 100 images are correct, accuracy is 80%. This is easy to read but can hide poor performance on rare classes. A model that always predicts a class making up 90% of a dataset can score 90% while failing every other class.

Precision, recall and macro F1

```
Precision = TP / (TP + FP)
Recall = TP / (TP + FN)
F1 = 2 × precision × recall / (precision + recall)
Macro F1 = unweighted mean of the per-class F1 scores
```

For a class, TP means correctly predicted members, FP means other classes incorrectly predicted as this class, and FN means members missed by the model. Precision asks “of the images called this class, how many were right?” Recall asks “of its true images, how many were found?”

Worked example for one class	Value
TP=8, FP=2, FN=4	8 correct members; 2 false alarms; 4 missed members.
Precision / recall / F1	0.800 / 0.667 / 0.727

Macro F1 gives each included class equal importance instead of letting frequent classes dominate by sample count. It is reported on a 0–1 scale, unlike the project’s percentage accuracy. The code does not pass an explicit full label list to f1_score: classes absent from both truth and predictions are omitted from that average.

Confusion matrices and degradation curves

Confusion-matrix rows are true classes; columns are predicted classes. Diagonal entries are correct predictions; off-diagonal entries show confusions. Plots include the full configured class range. Degradation curves show accuracy versus severity for each corruption; not every curve is guaranteed to decrease monotonically.

What is saved?

The evaluator writes a 70-row CSV with Dataset, Noise_Type, Severity, Accuracy and Macro_F1; a clean confusion-matrix PDF; 14 severity-5 confusion matrices; and a degradation-curve PDF. Clean metrics are printed and returned, not saved as a separate clean-metrics CSV. The checkpoint is saved after training, before target adaptation.

Separate an example from a result
All numerical examples in this guide are teaching examples, not newly measured benchmark scores. Existing result files need source/checkpoint provenance; generating this document did not reproduce a full experiment.

14 / How the code fits together

File / entry point	Responsibility
cli.py:17 / :36 / :68	Discover model folders, import the chosen class, parse arguments and launch evaluation.
dataset_loader.py:38	Load selected splits and native labels; resize, tensorize and cache images.
models/baseline/model.py:18	Baseline architecture, supervised training, prediction and checkpoint loading.
models/adabn/model.py:23	BN architecture and target-statistic adaptation at adapt_bn():181.
models/aco/model.py:25 / :131	Ant-colony optimizer and channel-gated CNN.
noise.py:36 / :105	Generate/cache corruptions and expose corrupted examples through a Dataset wrapper.
eval.py:39 / :237	Coordinate loading/training, clean scoring, adaptation and 70-condition scoring.
scripts/check_training.py	Focused assert-based regression checks, not a complete benchmark.

Classes, not two layer functions

--model baseline chooses the Baseline class; --model adabn chooses AdaBN; --model aco chooses ACOModel. Each is a separate nn.Module with its own layer definitions and trained state. The CLI dynamically imports the chosen class; it does not convert baseline weights into an AdaBN network.

Shared model interface

The constructor accepts num_classes. forward(images) returns logits. train(True/False) toggles mode, while train(dataset, ...) runs supervised training. test(dataset_or_loader) returns predictions and labels. load_weights(path) reports whether weights loaded. The evaluator detects optional adapt_bn() or adapt() methods for corrupted scoring.

Recent fixes represented in this guide

ReLU: baseline and ACO activations were changed; AdaBN already used ReLU.
PIL cache: stores the image itself, so repeated access returns (image, label), not a nested tuple.
ACO crash: mode switching now acts on the parent model, not the bound feature/classifier methods. A finally block restores training mode.
Checkpoints: class-count checks reject incompatible output heads; --retrain bypasses checkpoint loading. AdaBN metadata names the selected dataset.

The guide does not change the experiment
This PDF generation adds documentation only. It does not retrain models, fix remaining data-validation issues, upload a notebook to Colab, or synchronize a previously uploaded Drive notebook.

15 / Running each model and reading checkpoints

Run commands in PowerShell from the project root. These use the existing default output/<model>/<dataset> layout; no extra experiment folder is required.

```
cd D:\python\ML-Project

python cli.py --model baseline --dataset gtsrb --retrain --epochs 5

python cli.py --model adabn --dataset gtsrb --retrain --epochs 5

python cli.py --model aco --dataset gtsrb --retrain --epochs 5
```

Replace gtsrb with btsd or ctsd to train that model on a different dataset. Each command trains one model, then evaluates clean images and all 70 corrupted conditions. There is no separate train-only CLI switch in the current implementation.

Run all nine model/dataset combinations

```
foreach ($model in @("baseline", "adabn", "aco")) {

foreach ($dataset in @("gtsrb", "btsd", "ctsd")) {

python cli.py --model $model --dataset $dataset --retrain --epochs 5

if ($LASTEXITCODE -ne 0) { throw "Failed: $model on $dataset" }

}

}
```

Why --retrain matters

Without --retrain, the evaluator searches for an existing checkpoint and skips training when one loads. Merely changing --epochs does not force a new run. Search includes the dataset-specific folder and older model-level locations. --weights selects an explicit checkpoint and cannot be combined with --retrain.

What a checkpoint contains

The .pth file contains state_dict: learned parameters and registered buffers, including BN statistics or the ACO gate. It does not contain optimizer state, a full training history, or a resumable ACO search. Loading weights for evaluation is not the same as resuming an interrupted Adam run.

Old GELU weights are not equivalent ReLU models
Activation functions have no weight tensors, so an old GELU checkpoint can pass shape checks in the new ReLU network. Class-count checks cannot detect this semantic mismatch. Retrain for a valid current-architecture experiment.

Operational cautions

--retrain overwrites matching checkpoint/result paths in the default folder. Back up results you need before rerunning. Keep --target-size 49. ACO needs at least two epochs. The --workers option affects corrupted-data loading; --workers 0 avoids multiprocessing but may be slower. Training plus 70 conditions can take substantial time, especially on CPU.

16 / Limits and a defensible experiment design

What the current implementation supports

It implements three custom CNN variants, clean supervised training, optional checkpoint reuse, a selected corruption benchmark, AdaBN statistic adaptation, and an experimental ACO gate. These are implemented mechanisms, not automatic evidence that one method is better.

Risk	Why it matters / what to do
BN architecture confound	Compare one BN checkpoint with/without adaptation to isolate AdaBN; keep the baseline as a separate reference.
CTSD related variants	Check original/variant relationships; group related samples before splitting to avoid optimistic test results.
One-batch ACO objective	A gate can fit its search batch. Use held-out validation and repeated runs before claiming general improvement.
Uncontrolled randomness	Seed initialization, shuffle, ACO and corruption draws; repeat runs and report variability.
No validation/model selection	Create a training-only validation split. Do not select epochs or settings using final test performance.
Target access differs	State explicitly that AdaBN reads unlabelled target images before scoring, unlike the frozen baseline/ACO.
Runtime edge cases	Empty data, malformed inputs and arbitrary training settings are not fully validated. Singleton BN batches can fail.

Recommended reporting sequence

1. Verify split independence and class counts.
2. Fix preprocessing, seeds and a validation protocol before tuning.
3. Train baseline, BN source-only, BN+AdaBN and ACO under comparable settings.
4. Evaluate clean and corrupted data using recorded conditions; preserve checkpoints and run settings.
5. Report per-condition accuracy/macro F1, variability and failure cases, not only a single average.

What was checked for this PDF

The generator reads current local split counts, instantiates all three models for 43/62/58 classes, checks output shapes and absence of GELU, and verifies parameter totals. It also checks PDF page count, text presence and text bounds after rendering. The separate check_training.py covers small synthetic training paths; it is not full-dataset validation.

What was not established

No full training run, 70-condition rerun, real-road deployment test, or claim of superior current ReLU benchmark performance is made by this documentation task. Historical metrics cannot be silently relabelled as results of the new activation.

17 / Glossary and source references

Topic	Plain-language meaning
Tensor / feature map	A numerical array / a spatial response grid produced by a learned filter.
Parameter / hyperparameter	A learned weight / a chosen setting such as learning rate or ant count.
Buffer / checkpoint	Saved non-gradient state / a file containing the network state_dict.
Batch / epoch	A group of examples / one pass through the training split.
Logit / argmax	An unnormalized class score / the index of the largest score.
Gradient / backpropagation	Sensitivity of loss to a parameter / computing these sensitivities by the chain rule.
Domain shift / adaptation	Changed data distribution / adjusting a model to that distribution.
Ablation / leakage	A controlled removal of one mechanism / unintended overlap that makes evaluation too optimistic.
Pheromone / heuristic	ACO's accumulated channel preference / its variance-based search hint.

Authoritative background

Li et al., Revisiting Batch Normalization for Practical Domain Adaptation — the AdaBN idea.
Ioffe and Szegedy, Batch Normalization — BN background.
Kingma and Ba, Adam: A Method for Stochastic Optimization — optimizer background.
bethgelab/imagecorruptions — the corruption generator used here.

Implementation details were checked against local source and installed PyTorch docstrings. The ACO procedure is this repository's custom implementation, not a claimed reproduction of a particular paper. Installed PyTorch: 2.5.1+cu121.

Source fingerprint

SHA-256 prefixes identify the exact local Python content inspected. If these files change, regenerate the guide with python scripts/build_architecture_pdf.py. PDF generation needs ReportLab and PyMuPDF in addition to the project dependencies; it does not add them to the training requirements.

cli.py 789e5b2dea3e360d

dataset_loader.py a632122451d9e6cc

noise.py f52f74c00076a641

eval.py dd066032ee4a6e13

models/baseline/model.py 0f25754ae52490a5

models/adabn/model.py 036e245fe3113dc6

models/aco/model.py 49ea560bd05fddb7